50.043 Database Systems
Project Lab 3 Report

Class: CI01
Names: Tan Li Hon (1008920) & Celeste Chan Shi Ting (1008994)

Design Decisions

LockManager inner class | BufferPool.java

A separate LockManager inner class inside BufferPool was created to handle the locking logic. This will keep locking concerns separate from page caching and it maintains 3 data structures. sharedLocks will map each PageId to the set of transactions that hold shared locks on it, exclusiveLocks will map each PageId to the single transaction that holds an exclusive lock and waitingFor will map each transaction to the set of pages it is waiting to lock currently.

Lock Granularity | BufferPool.java

Each lock covers one page only rather than the entire table or tuple. Page-level locking will provide a good balance between concurrency and overhead calls. Therefore, as recommended, locking is implemented at page granularity.

Page-level vs tuple-level locking | BufferPool.java

Moving to tuple-level locking would mean keying sharedLocks and exclusiveLocks on RecordId rather than PageId, and holdersBlocking() would derive waits-for edges from tuple identities instead of pages. The benefit is higher concurrency, since two transactions writing different tuples on the same page would no longer block each other, which is important for small tuples where many share a page. The cost is a much larger lock table, since the number of entries scales with tuples rather than pages, and more bookkeeping on every acquire and release. Lock acquisition would also not be centralised in getPage() anymore, as fetching a page would not imply any tuple lock, so SeqScan, Insert and Delete would each need to request locks on the specific tuples they touch. Deadlock detection would additionally become more expensive as the waits-for graph grows with the finer granularity.

Lock Acquisition | getPage() in BufferPool.java

Locking is centralized in getPage() method instead of adding lock calls to each operator. Since all data access goes through getPage() method, this ensures all read and write are protected without the modification of SeqScan/Insert/Delete and other operators.

Non-Trivial Parts

Shared and Exclusive Lock Logic | acquireSharedLock() & acquireExclusiveLock() in BufferPool.java

For shared locks, transaction must wait if any other transaction is holding an exclusive lock, otherwise it can grant the shared lock. For exclusive locks, the transaction must wait if anyone else is holding any lock, otherwise it can grant an exclusive lock. For lock upgrade, if transaction already holds the only shared lock on the page and wants to write it can upgrade to exclusive lock instantly. This will avoid a deadlock occurrence. When a lock cannot be granted, blocking occurs using wait() and notifyAll() methods. The thread will call wait(50) and block for 50ms before trying again. When lock is released, notifyAll() will wake up the threads waiting so they can try to acquire the locks again.

No steal eviction | evictPage() method in BufferPool.java

The eviction policy must adhere to the no steal constraint. Thus evictPage() method iterates through the pages in LRU order and skips dirty pages. If isDirty() == null means that all pages are safe to evict. Thus, if all pages are dirty, a DbException will be thrown since eviction will be impossible without violating no steal constraint.

Cycle detection over the waits-for graph | hasDeadlock() & dfs() in BufferPool.java

Deadlock detection is implemented by searching for cycles in a waits-for graph rather than using a timeout policy. The graph is not stored explicitly. Instead, its edges are derived on demand from the existing lock tables

by a holdersBlocking() helper, which takes a transaction, looks up the pages it is waiting on in waitingFor, and returns every other transaction holding a shared or exclusive lock on those pages. hasDeadlock() expands the requesting transaction's immediate neighbours and calls dfs() on each; dfs() recurses through the graph and returns true if it reaches the original transaction, which means a cycle exists. A visited set is shared across all branches so each transaction is expanded at most once, and the starting transaction is deliberately excluded from that set so it remains reachable as the search target. Detection is invoked inside both acquire loops immediately after a transaction registers itself in waitingFor, so a cycle is caught at the moment it forms rather than after a delay. The transaction that detects the cycle aborts itself by throwing TransactionAbortedException, which breaks the cycle and allows the remaining transactions to proceed. Scanning sharedLocks in addition to exclusiveLocks is essential because in a lock-upgrade deadlock, both transactions hold shared locks on the same page and each waits for the other to release, so an implementation that only followed exclusive-lock holders would find no edges and hang.

The main advantage over a timeout policy is precision. A timeout cannot distinguish a genuine deadlock from a transaction that is merely slow, so it aborts transactions that would have completed. Cycle detection only aborts when a cycle probably exists, and detects it immediately rather than after a fixed wait. The cost is that the graph must be traversed on every blocked lock request, which is more expensive per request than checking a clock, and the traversal happens while holding the LockManager monitor. Compared to prevention schemes such as wait-die or wound-wait, which order transactions by timestamp and abort preemptively, detection permits more transactions to proceed but requires the graph machinery to be maintained.

Flushing only the transaction's own dirty pages | flushPages() in BufferPool.java

flushPages(tid) retrieves the set of pages the transaction holds locks on via getPagesHeldBy(), then flushes only those pages whose dirtying transaction matches tid. The tid.equals(page.isDirty()) check matters because a transaction may hold a read lock on a page that another transaction dirtied; flushing that page would write another transaction's uncommitted changes to disk and violate the no steal constraint. Restricting the flush to locked pages rather than scanning the entire cache also keeps the cost proportional to the transaction's footprint.

Commit and abort handling | transactionComplete() in BufferPool.java

The two-argument transactionComplete(tid, commit) branches on the commit flag. On commit it calls flushPages(tid) to force the transaction's dirty pages to disk, giving a FORCE policy that removes the need for redo logging. On abort it iterates the pages held by the transaction and calls discardPage() on any it dirtied, dropping the modified copy from the cache so the next getPage() re-reads the clean version from disk. This restores the on-disk state without needing an undo log. Both paths converge on releaseAllLocks(tid), which is called outside the branch so locks are released regardless of outcome. The single-argument transactionComplete(tid) simply delegates to transactionComplete(tid, true), as commit is the default behaviour.

API Changes

No changes made to API.

Missing or Incomplete Elements

1. Deadlock resolution always aborts the transaction that detects the cycle, rather than selecting a victim by age, number of locks held, or work done. This is correct but may not abort the cheapest transaction.
2. Lock acquisition polls with wait(50) rather than being woken precisely when the required lock is released, so a waiting transaction may be delayed by up to 50ms after the lock becomes available.
3. No logging or recovery is implemented. Durability relies on flushing dirty pages at commit, so a crash mid-commit could leave the database inconsistent.
4. B+ tree operations in BTreeFile.java are not implemented, so BTreeDeadlockTest does not pass. This is not required for Lab 3.